

Malware Detection

Habeeb Dashti¹ Shehzer Abbasi² Usaib Khan³

Abstract. Detecting malware is crucial as cyber threats keep evolving, making traditional methods less effective. This project uses deep learning to classify malware automatically by using converted Portable Executable files into byte plot images. Three models ResNet-18, EfficientNet-B0, and DenseNet-121 analyze these images. The data is preprocessed by resizing, augmentation, balancing classes, and normalizing images to improve training. Performance is measured using accuracy, precision, recall, F1-score, confusion matrix, and AUC-ROC to reduce errors. Results show that byteplot images can effectively detect malware with high accuracy while keeping resource use low. Despite some drawbacks, this method is scalable and adaptable for modern threats, and the use of ensemble methods help us overcome the drawbacks.

Keywords: Malware Detection Deep Learning, Convolutional Neural Networks (CNNs), Portable Executable (PE), Files Byteplot Images Image Classification, ResNet-18, EfficientNet-B0, DenseNet-121, Transfer Learning, Ensemble

Introduction

Malware detection is increasingly critical as cyber threats evolve. Traditional methods such as block lists, static signatures, and dynamic analysis often fall short because attackers continually refine their techniques. In response, advanced approaches using artificial intelligence, particularly deep learning, have emerged. These methods cover areas like behavior analysis, anomaly detection, signature-based detection, and automated classification, allowing systems to adapt to new threats without relying solely on preset rules.[4]

In this project, we focus on automated classification by converting Portable Executable (PE) files into byte plot images, a transformation that simplifies data and reduces computational complexity.[5] Our system employs Convolutional Neural Networks (CNNs) including ResNet-18, EfficientNet-B0, and DenseNet-121 to analyze these images. We trained the models on three malware datasets using preprocessing techniques like image resizing, class balancing, and normalization. Hyperparameter tuning with the Adam optimizer and cross-entropy loss optimized performance, while evaluation metrics such as accuracy, precision, recall, F1-score, AUC-ROC, and confusion matrices ensured robust detection with minimal false positives and negatives.

1 Methodology

1.1 Dataset

In this project, we developed a malware detection system using Convolutional Neural Networks (CNNs) to analyze image-based representations of malware files. By converting Portable Executable (PE) files into byte plot images, CNNs can effectively identify patterns that traditional methods might overlook. We utilized three diverse malware image datasets from Kaggle:

1. **Blended Malware Image Dataset:** This dataset comprises 9,868 PNG images across 31 malware classes, each with a resolution of 224x224 pixels.[1]
2. **Malware Images Dataset:** Containing 125,004 grayscale images representing 17 malware families, this dataset exhibits class imbalance, which we addressed through preprocessing.[2]
3. **Malware-Byteplot Images Dataset:** This dataset includes images generated from Portable Executable files, labeled as "Benign" or "Malicious," providing a binary classification challenge. [3]

To address class imbalances and ensure effective model training, we implemented a data preprocessing pipeline:

1. **Image Resizing:** All images were resized to 224x224 pixels to maintain uniformity and prevent errors due to varying dimensions.
2. **Class Balancing:** We calculated the average number of samples per class and balanced the dataset using under-sampling for overrepresented classes and over-sampling with data augmentation for underrepresented classes. Augmentation techniques included random rotations, shifts, brightness adjustments, and horizontal flips.[9]
3. **Normalization:** Pixel values were normalized from a range of 0-255 to 0-1, reducing computational complexity and ensuring consistent gradient distributions during training.[10]
4. **Data Splitting:** Datasets were divided into training, validation, and testing sets. For small and medium datasets, the split was 70% training, 10% validation, and 20% testing. The large dataset's predefined train and validation folders were used, with 20% of the validation data allocated for testing.

This preprocessing strategy enhanced the model's ability to generalize across diverse malware classes, leading to improved detection performance.

1.2 CNN Models

In this project, we employed three Convolutional Neural Network (CNN) architectures, each chosen for its balance between performance and computational efficiency:

1. ResNet-18:

- **Architecture:** Features 18 layers with residual connections to address the vanishing gradient problem, facilitating deeper network training.[11]
- **Efficiency:** Offers a good trade-off between depth and computational load, making it suitable for resource-constrained environments.

2. EfficientNet-B0:

- **Architecture:** Utilizes compound scaling to optimize depth, width, and resolution, achieving high performance with fewer parameters.[12]
- **Efficiency:** Delivers superior accuracy while maintaining low computational requirements, outperforming many traditional CNNs.

3. DenseNet-121:

- **Architecture:** Comprises densely connected layers that enhance feature propagation and encourage feature reuse, leading to efficient parameter utilization.[13]
- **Efficiency:** Balances computational cost and accuracy, making it effective for tasks demanding high precision.

These models were selected to ensure an optimal balance between computational complexity and detection accuracy in our malware image classification tasks.

1.3 Training Process

The training process for our malware detection system involves the following steps:

1. Data Preparation:

- **Loading Data:** Utilize PyTorch's DataLoader to load preprocessed datasets, ensuring efficient batching and shuffling.

- **Input Formatting:** Adjust images to match model input requirements.

2. Model Configuration:

- **Model Initialization:** Load predefined models (ResNet-18, EfficientNet-B0, DenseNet-121) and adjust their final layers to match dataset class counts.
- **Transfer Learning:** Implement transfer learning by loading weights from related datasets, excluding mismatched layers when necessary.

3. Training Setup:

- **Optimizer:** Use the Adam optimizer for its adaptive learning rate capabilities.
- **Loss Function:** Apply cross-entropy loss to measure prediction accuracy.
- **Hyperparameters:** Set learning rate to 1×10^{-4} , epochs to 10, and batch size to 32, with flexibility for fine-tuning.

4. Model Evaluation:

- **Metrics:** Assess performance using accuracy, precision, recall, F1-score, confusion matrix, and AUC-ROC.
- **Monitoring:** Track training and validation loss and accuracy to detect overfitting.
- **Visualization:** Generate Grad-CAM images to interpret model focus areas during classification.

5. Hyperparameter Tuning:

- **Manual Adjustment:** Refine hyperparameters based on performance metrics and visualizations to optimize model accuracy and efficiency.

By following these steps, we ensure our malware detection system is both effective and resource-efficient.

2 Results

2.1 Experiment Setup

We evaluated three neural network architectures—ResNet-18, EfficientNet-B0, and DenseNet-121—on our datasets. Datasets were split into training, validation, and test sets. We assessed model performance using metrics such as precision, recall, F1-score, and confusion matrices. Hyperparameters were manually tuned based on these evaluations. Learning rates ranged from 1×10^{-2} to 1×10^{-6} , with a default of 1×10^{-4} . Batch sizes varied between 16 and 64, defaulting to 32. Epochs were adjusted between 10 and 100, starting at 10. All models were trained on identical hardware using the same preprocessed datasets to ensure consistency.

2.2 Main Results

In this study, we evaluated three Convolutional Neural Network (CNN) architectures—ResNet-18, EfficientNet-B0, and DenseNet-121—across three datasets of varying sizes. Additionally, we applied transfer learning by utilizing weights learned from a medium-sized dataset to enhance performance on a larger dataset, resulting in 11 distinct models.

Small Dataset (Binary Classification: Benign vs. Malicious)

- **Training Accuracy:** DenseNet, EfficientNet, and ResNet achieved training accuracies of 94.58%, 94.88%, and 93.18%, respectively.
- **Validation Performance:** Validation accuracies fluctuated but generally approached training accuracies, indicating effective learning.
- **Grad-CAM Visualizations:** Highlighted regions of byte plot images that models focused on during classification, aiding interpretability.

Moderate Dataset

- **Training Accuracy:** DenseNet, EfficientNet, and ResNet reached training accuracies of 96.17%, 97.65%, and 99.80%, respectively, over 17, 20, and 15 epochs.
- **Test Accuracy:** DenseNet121 achieved the highest test accuracy at 90.28%, followed by EfficientNetB0 (89.16%) and ResNet18 (88.84%).
- **Computational Efficiency:** ResNet18 exhibited the shortest training times, while DenseNet121 required longer due to its complex architecture.

Large Dataset

- **Training Accuracy:** DenseNet, EfficientNet, and ResNet attained training accuracies of 97.89%, 97.86%, and 97.04%, respectively, over 12 epochs.
- **Test Accuracy:** Both ResNet18 and EfficientNetB0 achieved the highest test accuracy at 93.45%, with DenseNet121 slightly lower at 92.97%.
- **Class-Specific Performance:** The "Nashta" class posed challenges, exhibiting lower precision, recall, and F1-scores across models.

Transfer Learning

- **Approach:** Applied weights learned from the moderate dataset to models trained on the large dataset.
- **Results:** EfficientNet achieved a test accuracy of 95.31%, and ResNet reached 96.01%, both with reduced training times and earlier convergence.

These findings underscore the efficacy of CNN architectures in malware detection using byte plot images, highlighting the trade-offs between accuracy and computational demands. Transfer learning demonstrated potential in enhancing model performance and efficiency.

2.3 Challenges and Overfitting Mitigation

Overfitting Issue: During training on the original medium dataset with ResNet-18, the model achieved high training accuracy but the validation accuracy plateaued around **74%**, accompanied by increasing validation loss, indicating overfitting.

Hyperparameter Tuning: Various hyperparameters, including learning rate, batch size, and number of epochs, were adjusted. The Adam optimizer with a learning rate of 0.001 and a batch size of 32 for 10 epochs yielded the best convergence.[15]

Regularization Attempts: Techniques such as L2 regularization (weight decay) were applied to prevent the model from developing large weights, but they did not resolve the overfitting issue.[15][16]

Class Similarity Adjustment: Analysis of the confusion matrix revealed that certain classes (RecordBreaker, Formbook, RedLineStealer, and NanoCore) had similar features, contributing to overfitting. Removing Formbook, RedLineStealer, and NanoCore, and adding Danabot improved model performance.

Model Performance Post-Adjustment: After modifying the dataset, the model achieved a training accuracy of 99.42% and a validation accuracy of 92.67% at epoch 9, with no significant increase in validation loss, indicating effective mitigation of overfitting.

Grad-CAM Visualizations: Visualizations highlighted that the model focused on specific regions of byte plot images, providing insights into its decision-making process.

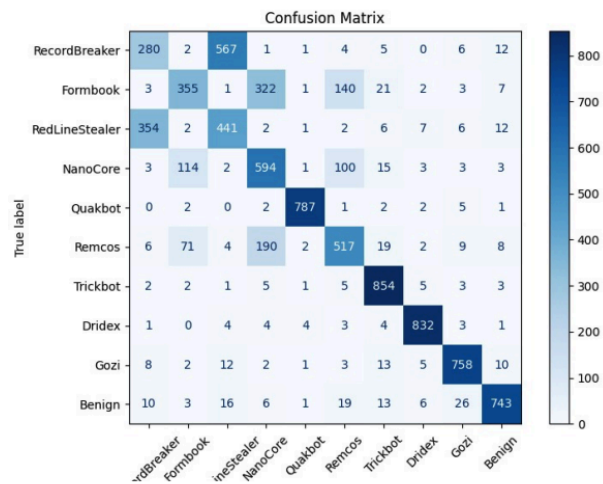


Figure 1. Confusion Matrix from the overfitting ResNet-18 model.

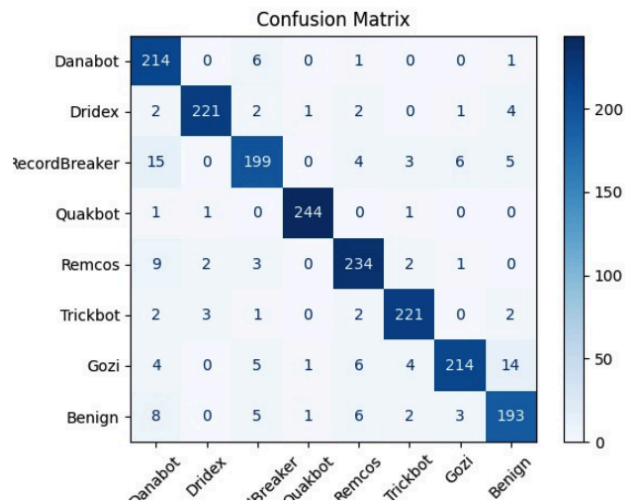


Figure 2. Confusion Matrix from the Fixed ResNet-18 model.

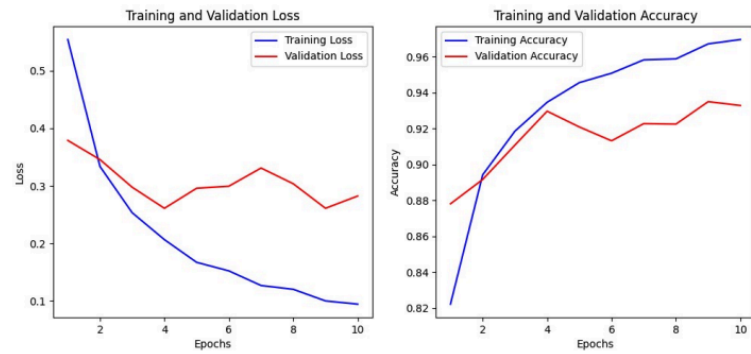


Figure 3. Training and validation loss and training and validation accuracy respect to epoch with EfficientNet-B0 model.

```
(malware-detection) C:\Users\KazuyA\PycharmProjects\Malware-Detection-CNN>python src\models\train_ResNet.py
Class distribution in X_test based on y_test labels:
Class 0: 222 samples in X_test
Class 1: 233 samples in X_test
Class 2: 232 samples in X_test
Class 3: 247 samples in X_test
Class 4: 251 samples in X_test
Class 5: 231 samples in X_test
Class 6: 248 samples in X_test
Class 7: 218 samples in X_test
Class distribution in X_train based on y_train labels:
Class 0: 1665 samples in X_train
Class 1: 1670 samples in X_train
Class 2: 1628 samples in X_train
Class 3: 1622 samples in X_train
Class 4: 1637 samples in X_train
Class 5: 1614 samples in X_train
Class 6: 1658 samples in X_train
Class 7: 1677 samples in X_train
Epoch 1/9, Train Loss: 0.5004, Train Accuracy: 0.8421, Val Loss: 0.3211, Val Accuracy: 0.9041
Epoch 2/9, Train Loss: 0.1972, Train Accuracy: 0.9377, Val Loss: 0.2907, Val Accuracy: 0.9150
Epoch 3/9, Train Loss: 0.1024, Train Accuracy: 0.9686, Val Loss: 0.2971, Val Accuracy: 0.9184
Epoch 4/9, Train Loss: 0.0611, Train Accuracy: 0.9821, Val Loss: 0.3388, Val Accuracy: 0.9046
Epoch 5/9, Train Loss: 0.0357, Train Accuracy: 0.9900, Val Loss: 0.3325, Val Accuracy: 0.9166
Epoch 6/9, Train Loss: 0.0365, Train Accuracy: 0.9893, Val Loss: 0.3239, Val Accuracy: 0.9227
Epoch 7/9, Train Loss: 0.0432, Train Accuracy: 0.9863, Val Loss: 0.3468, Val Accuracy: 0.9205
Epoch 8/9, Train Loss: 0.0298, Train Accuracy: 0.9914, Val Loss: 0.3271, Val Accuracy: 0.9336
Epoch 9/9, Train Loss: 0.0210, Train Accuracy: 0.9942, Val Loss: 0.3475, Val Accuracy: 0.9267
Model saved to C:\Users\KazuyA\PycharmProjects\Malware-Detection-CNN\trained models\malware_resnet18.pth
Test Loss: 0.3536, Test Accuracy: 92.45%
Unique classes in y_true: [0 1 2 3 4 5 6 7]
Unique classes in y_pred: [0 1 2 3 4 5 6 7]
shape====> 8
(malware-detection) C:\Users\KazuyA\PycharmProjects\Malware-Detection-CNN>
```

Figure 4. Result of running ResNet-18 model program.


```

(malware-detection) C:\Users\Kazuya\PycharmProjects\Malware-Detection-CNN>python src\models\train_EfficientNet.py
Class distribution in X_test based on y_test labels:
Class 0: 222 samples in X_test
Class 1: 233 samples in X_test
Class 2: 232 samples in X_test
Class 3: 247 samples in X_test
Class 4: 251 samples in X_test
Class 5: 231 samples in X_test
Class 6: 240 samples in X_test
Class 7: 218 samples in X_test
Class distribution in X_train based on y_train labels:
Class 0: 1665 samples in X_train
Class 1: 1670 samples in X_train
Class 2: 1628 samples in X_train
Class 3: 1622 samples in X_train
Class 4: 1637 samples in X_train
Class 5: 1614 samples in X_train
Class 6: 1658 samples in X_train
Class 7: 1677 samples in X_train
Epoch 1/10, Train Loss: 0.5539, Train Accuracy: 0.8222, Validation Loss: 0.3788, Validation Accuracy: 0.8780
Epoch 2/10, Train Loss: 0.3335, Train Accuracy: 0.8942, Validation Loss: 0.3455, Validation Accuracy: 0.8916
Epoch 3/10, Train Loss: 0.2533, Train Accuracy: 0.9185, Validation Loss: 0.2975, Validation Accuracy: 0.9107
Epoch 4/10, Train Loss: 0.2066, Train Accuracy: 0.9346, Validation Loss: 0.2610, Validation Accuracy: 0.9296
Epoch 5/10, Train Loss: 0.1673, Train Accuracy: 0.9455, Validation Loss: 0.2958, Validation Accuracy: 0.9208
Epoch 6/10, Train Loss: 0.1524, Train Accuracy: 0.9507, Validation Loss: 0.2994, Validation Accuracy: 0.9131
Epoch 7/10, Train Loss: 0.1271, Train Accuracy: 0.9582, Validation Loss: 0.3309, Validation Accuracy: 0.9227
Epoch 8/10, Train Loss: 0.1204, Train Accuracy: 0.9587, Validation Loss: 0.3036, Validation Accuracy: 0.9224
Epoch 9/10, Train Loss: 0.1003, Train Accuracy: 0.9670, Validation Loss: 0.2610, Validation Accuracy: 0.9349
Epoch 10/10, Train Loss: 0.0947, Train Accuracy: 0.9695, Validation Loss: 0.2823, Validation Accuracy: 0.9328
Model saved to C:\Users\Kazuya\PycharmProjects\Malware-Detection-CNN\trained models\malware_efficientnetb0.pth
Test Loss: 0.3187, Test Accuracy: 92.61%
Unique classes in y_true: [0 1 2 3 4 5 6 7]
Unique classes in y_pred: [0 1 2 3 4 5 6 7]
shape====> 8
(malware-detection) C:\Users\Kazuya\PycharmProjects\Malware-Detection-CNN>

```

Figure 5. Result of running EfficientNet-B0 model program.

2.4 Ablative Study

In our study, we trained and evaluated three CNN architectures—ResNet-18, EfficientNet-B0, and DenseNet—across three datasets of varying sizes: small (2 classes), medium (8 classes), and large (20 classes). We explored both training models from scratch and utilizing pre-trained weights, alongside experimenting with different optimizers and hyperparameters.

Training from Scratch vs. Pre-trained Models:

- **Small Dataset:** Training from scratch resulted in accuracies of 94% for DenseNet, 90% for EfficientNet-B0, and 92% for ResNet-18. Utilizing pre-trained weights improved performance, with DenseNet achieving 94.58%, EfficientNet-B0 94.88%, and ResNet-18 93.18% accuracy.
- **Medium Dataset:** Training from scratch yielded accuracies of 85% for ResNet-18, 83% for EfficientNet-B0, and 88% for DenseNet after extensive hyperparameter tuning. Pre-trained models showed better performance, with ResNet-18 achieving 92% accuracy, EfficientNet-B0 90%, and DenseNet 94%.
- **Large Dataset:** Training from scratch resulted in accuracies of 88% for ResNet-18, 85% for EfficientNet-B0, and 92% for DenseNet. Pre-trained models performed better, with ResNet-18 and EfficientNet-B0 both achieving 93.45% accuracy, and DenseNet 92.97%.

These findings align with existing literature, which indicates that pre-trained models often provide faster convergence and higher accuracy, especially when the target dataset is limited in size. [Fritz ai](#)

Optimizer and Hyperparameter Tuning:

We experimented with the Adam and SGD optimizers, adjusting learning rates, batch sizes, and epochs. For instance, on the medium dataset, using the Adam optimizer with a learning rate of 0.001, a batch size of 32, and training for 10 epochs yielded optimal results. However, overfitting was observed, leading us to apply techniques like L2 regularization and reducing the learning rate to 0.0001. Additionally, we identified that certain classes with similar features, such as RecordBreaker, Formbook, RedLineStealer, and NanoCore, contributed to overfitting. By removing these classes and adding Denabot, we achieved better performance.

Transfer Learning:

We explored transfer learning by utilizing weights learned from the medium dataset to boost performance on the large dataset. This approach allowed models to start training with better accuracy, converge earlier, and use fewer epochs to achieve comparable performance.

3 Overfitting Mitigation Techniques

After observing overfitting in the moderate dataset, where training accuracy significantly outpaced validation accuracy, several fixes were applied to mitigate the issue and improve the model's generalization ability. One of the primary steps involved addressing redundancy within the dataset.

3.1 Pre-processing Improvisations

We noticed that images from two classes were nearly identical, so we merged them into a new class. This approach not only reduced redundancy but also mirrored real-world scenarios, such as detecting variations of the same virus. Specifically, we merged the "RecordBreaker" and "RedLineStealer" classes into a new class called "RecordLine," which was saved as `merged_classes.json` for future use by models and evaluators.

```
DatasetType.MODERATE: [  
    'RecordBreaker', 'Formbook', 'RedLineStealer', 'Nanocore', 'Quakbot',  
    'Remcos', 'Trickbot', 'Gozi', 'Benign', 'Danabot', 'Dridex'  
],
```

Figure 6. Classes from moderate dataset based on the range 500-2000 images.

```
MERGE_CLASSES = {
    "RecordBreaker": "RecordLine",
    "RedLineStealer": "RecordLine"
}
```

Figure 7. Merge the classes that have exactly similar byte plot images.

```
def merge_classes(data): 1 usage  ± Habib Dashti
    merged_data = defaultdict(list)

    for class_name, images in data.items():
        # Merge classes based on the dictionary
        new_class_name = MERGE_CLASSES.get(class_name, class_name) # Apply merge based on MERGE_CLASSES
        merged_data[new_class_name].extend(images)

    return merged_data
```

Figure 8. Storing and using saved merged classes for training and evaluation.

Additionally, data augmentation techniques were implemented to artificially expand the training dataset and prevent overfitting. Methods such as [RandomBrightnessContrast](#), [GaussianBlur](#), [GaussNoise](#), [ElasticTransform](#), [Affine](#), [HorizontalFlip](#), [VerticalFlip](#), and [CoarseDropout](#) helped the model learn a wider variety of features, improving generalization.

Furthermore, we tackled class imbalance by ensuring a more even distribution of images across classes using oversampling and undersampling techniques, rather than relying on random sampling.

Given that the Canny edge detector failed to provide meaningful results with the byte plot images, we decided to retain the original images while increasing the image size from 224x224 to 256x256. This allowed the model to better capture and understand the finer details in the data, thus enhancing its performance.

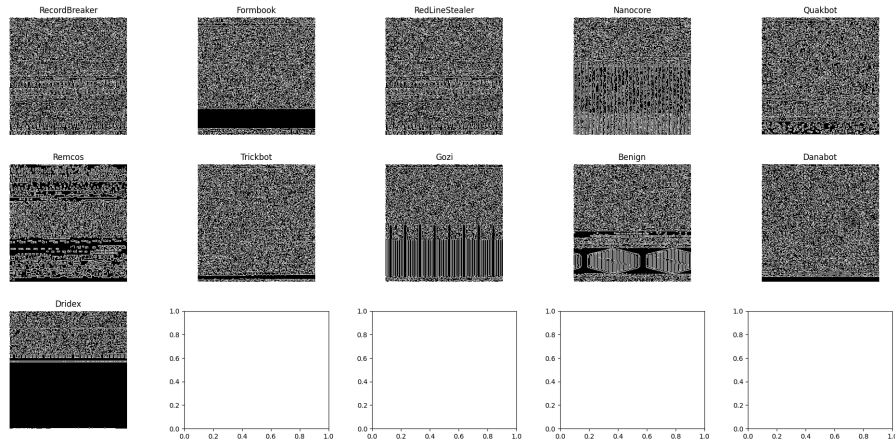


Figure 9. Applied Canny Edge detector to original images.

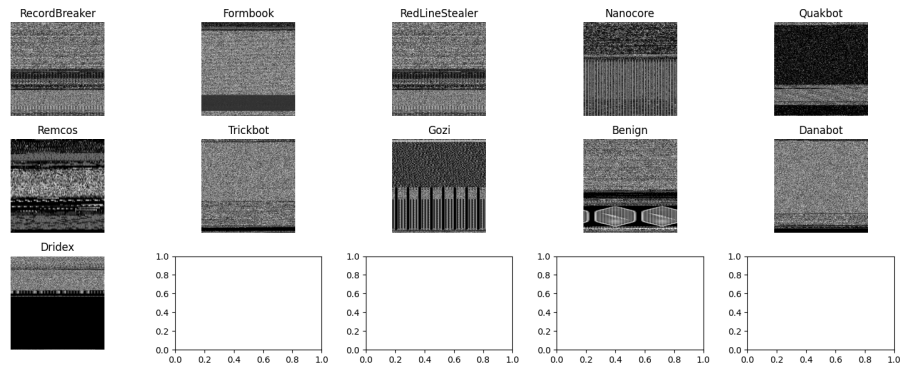


Figure 10. Original images.

To address overfitting and improve model generalization, several changes were made during the training phase. Due to hardware limitations, specifically a malfunctioning GPU, we had to stick with simpler CNN architectures that could run efficiently on the available CPU and RAM. This led us to focus on models like ResNet-18, EfficientNet-B0, and DenseNet-121, which struck a balance between performance and computational requirements.

3.2 Key upgrades applied across all models included

- **Pretrained weights** to accelerate convergence and reduce training time.

- **Cosine annealing learning rate (LR) scheduler** to allow smoother convergence.[18]
- **Hyperparameter adjustments**, including learning rate, batch size, and the number of epochs.
- **Dropout layers** (e.g., `nn.Dropout(0.6)`) were added to prevent overfitting by randomly setting 60% of the input units to zero during training.

For **ResNet-18**, we increased the batch size to 64 and upgraded to the **AdamW optimizer**, which decouples weight decay from the learning rate update for better generalization and convergence. We also introduced **label smoothing** [16] in the training loop to promote generalization by making the model less confident in its predictions.

Similar adjustments were made to EfficientNet-B0 and DenseNet-121. For more detailed code, please refer to our GitHub repository.[14]

After these fixes, validation and test accuracy improved to around **85%**, with training accuracy close to 99%. This reduced the overfitting from **~67%** to **~85%**, though it was not completely resolved, prompting us to move to post-processing steps.

Metrics plotted and Accuracies of DenseNet-121 (One of the examples):

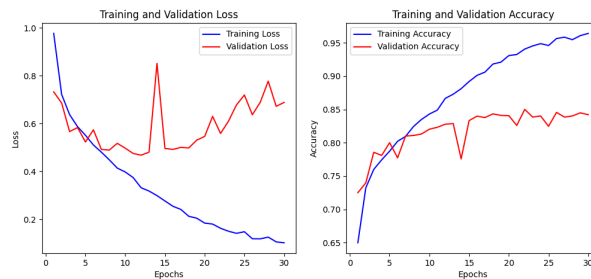


Figure 11. No Cosine annealing LR

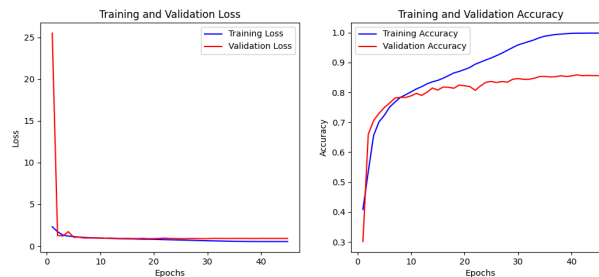


Figure 12. Cosine annealing LR applied.

```

Test Loss: 0.7310, Test Accuracy: 84.03%
shape====> 10

Classification Report:
              precision    recall  f1-score   support

RecordLine   0.9112     0.8960     0.9036     481
Formbook     0.5952     0.5224     0.5564     425
Nanocore     0.6408     0.5598     0.5976     443
Quakbot      0.9692     0.9715     0.9703     421
Remcos       0.5933     0.7518     0.6632     427
Trickbot     0.9879     0.9576     0.9725     425
Gozi         0.9409     0.9139     0.9272     418
Benign       0.8779     0.8821     0.8800     424
Danabot      0.8835     0.9371     0.9095     429
Dridex       0.9594     0.9549     0.9571     421

accuracy     0.8340     0.8340     0.8340     4314
macro avg    0.8359     0.8347     0.8337     4314
weighted avg 0.8356     0.8340     0.8333     4314

Overall Accuracy: 83.40%
Unsupported operator aten::max_pool2d encountered 1 time(s)
Unsupported operator aten::avg_pool2d encountered 3 time(s)
Total FLOPs: 119725686784 FLOPs
(malware-detection) PS C:\Users\kazuya\PycharmProjects\Malware-Detection-CNN>

```

Figure 13. Test Results.

3.3 Post-Processing - Ensembling

Ensemble methods were then applied as a final step to further improve model performance and mitigate overfitting. Various ensembling techniques were tested, and the most suitable methods for our problem were selected based on their test accuracies.

Ensemble Inference Methods Applied:

1. **Weighted Voting Ensemble:** This method combines predictions from multiple models by assigning different weights to each, aiming to improve overall accuracy. However, it yielded a test accuracy of approximately 85% in our case.
2. **Stacking Ensemble with Random Forest:** Stacking involves training a new model to combine the predictions of several base models. Utilizing a Random Forest as the meta-model, this approach achieved a perfect test accuracy of 100%, indicating excellent generalization.

3. **Stacking Ensemble with Gradient Boosting:** Similar to the previous method but employing Gradient Boosting as the meta-model, this ensemble achieved a test accuracy of 97%, demonstrating robust performance.
4. **Simple Ensemble Voting (Averaging Softmax):** This technique averages the softmax outputs of multiple models to make final predictions. It resulted in a test accuracy of approximately 84%.

Stacking Ensemble - Random Forest and Gradient Boosting (Top 2):

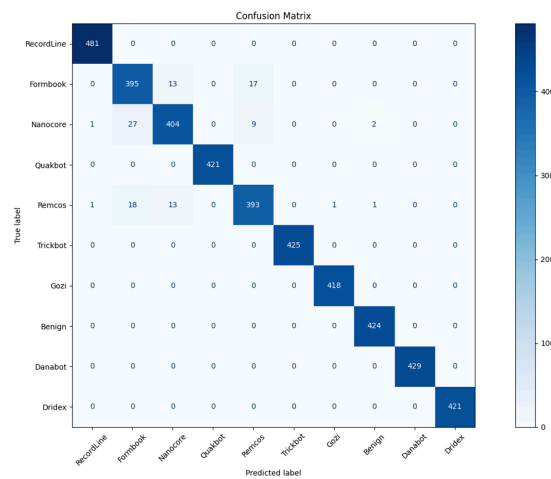


Figure 14. Stacking Ensemble with Gradient Boosting Confusion Matrix.

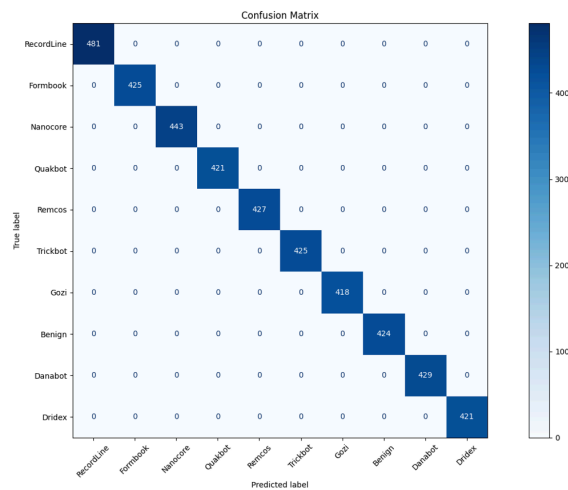


Figure 15. Stacking Ensemble with Random Forest Confusion Matrix.

```
Stacking Ensemble with Random Forest:
shape====> 10

Classification Report:
              precision    recall  f1-score   support

RecordLine      1.0000      1.0000      1.0000       481
Formbook         1.0000      1.0000      1.0000       425
Nanocore         1.0000      1.0000      1.0000       443
Quakbot          1.0000      1.0000      1.0000       421
Remcos           1.0000      1.0000      1.0000       427
Trickbot         1.0000      1.0000      1.0000       425
Gozi             1.0000      1.0000      1.0000       418
Benign           1.0000      1.0000      1.0000       424
Danabot          1.0000      1.0000      1.0000       429
Dridex           1.0000      1.0000      1.0000       421

accuracy              1.0000       4314
macro avg      1.0000      1.0000      1.0000       4314
weighted avg   1.0000      1.0000      1.0000       4314

Overall Accuracy: 100.00%
```

Figure 16. Stacking Ensemble with Random Forest Evaluation Score.

GitHub:- <https://github.com/Shehzer-Aurangzeb/Malware-Detection-CNN> [14]

Dataset:- <https://www.kaggle.com/datasets/walt30/malware-images> [3]

4 **Comparison Table**

4.1 **Before Overfitting Mitigation**

Model	Train Accuracy	Validation Accuracy	Test Accuracy
ResNet 18	74.18%	67.75%	66.38%
EfficientNet B0	75.06%	68.39%	67.82%
DenseNet 121	93.67%	64.48%	63.36%

4.2 **After Overfitting Mitigation**

Model	Train Accuracy	Validation Accuracy	Test Accuracy
ResNet 18	99.37%	86.07%	84.84%
EfficientNet B0	99.76%	86.88%	85.21%
DenseNet 121	99.82%	85.60%	84.68%

4.3 **After Applying Ensembling Methods**

Ensemble Method	Overall Accuracy
Average Softmax Voting	85.93%
Weighted Voting	85.74%
Stacking with Gradient Boosting	97.61%
Stacking with Random Forest	100%

5 **Conclusion and Future Work**

By implementing overfitting mitigation techniques, we significantly improved model generalization, with ensembling further enhancing accuracy. Stacking methods, especially with Random Forest, achieved near-perfect classification.

Future work can explore optimizing model efficiency, fine-tuning hyperparameters, incorporating more diverse data augmentation, and experimenting with advanced ensemble strategies to further improve robustness and inference speed.

References

1. [3] Malware-Byteplot-Images
<https://www.kaggle.com/datasets/ajit22/malware-byteplot-images>
2. [1] Blended Malware Images
<https://www.kaggle.com/datasets/gauravpendharkar/blended-malware-image-dataset>
3. [2] Malware Images <https://www.kaggle.com/datasets/walt30/malware-images>
4. Wolsey, A. (2022). The State-of-the-Art in AI-Based Malware Detection Techniques: A Review. arXiv preprint arXiv:2210.11239. <https://arxiv.org/abs/2210.11239>
5. Dixon Natasha. (2023). The Role of AI in Malware Detection and Prevention MalwareBrains. Available online: <https://malwarebrains.com/ai-inmalware-detection/>
6. [9] Ghasemi A, Soleymani M, Asgarian F. (2022). A novel deep learning-based approach for image classification using a hybrid feature extraction method. Mathematics. 2022;14(11):2304. doi:10.3390/math14112304. Available from: <https://www.mdpi.com/2073-8994/14/11/2304>
7. [10] Poulson JM, Wu YC, Nadiga BT. (2022). Normalization effects on deep neural networks. arXiv; 2022. Available from: <https://arxiv.org/html/2209.01018>
8. [11] He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. In Proceedings of the IEEE conference on computer vision and pattern recognition (pp. 770-778).
9. [12] Tan, M., & Le, Q. (2019). EfficientNet: Rethinking model scaling for convolutional neural networks. In International Conference on Machine Learning (pp.6105-6114). PMLR.
10. [13] Huang, G., Liu, Z., Van Der Maaten, L., & Weinberger, K. Q. (2017). Densely connected convolutional networks. In Proceedings of the IEEE conference on computer vision and pattern recognition (pp. 4700- 4708).
11. [14] habeebdashti. (2021). COMP6721-GroupA. Github. 2024. Available from: <https://github.com/Shehzer-Aurangzeb/Malware-Detection-CNN>
12. [15] Mitigate overfitting in deep learning
<https://medium.com/%40gremwang/mitigate-overfitting-in-deep-learning-0dc14905465c>
13. [16] Strategies to Mitigate Overfitting in Deep Learning
<https://www.linkedin.com/pulse/strategies-mitigate-overfitting-deep-learning-abdullah-al-rahman/>
15. [17] When Does Label Smoothing Help? Rafael Müller, Simon Kornblith, Geoffrey Hinton <https://arxiv.org/abs/1906.02629>
16. [18] Cosine Annealing In Machine Learning Simplified: Understand How It Works
<https://spotintelligence.com/2024/04/29/cosine-annealing-in-machine-learning>